Project Title: Library Management System

Course Name: Problem Solving using C++

Student Name: Nikhil Kumawat & Vaidika Shivhare

Roll Number: PU02425EUG11011 PU02425EUG11035

Class / Section: BCA 1st Year

Faculty Name: Sakshi Verma Ma’am

Institute Name: Avantika University

Academic Year: 2025–26

CERTIFICATE

This is to certify that the project titled “Library Management System” has been successfully completed by Nikhil Kumawat and Vaidika Shivhare under the guidance of Sakshi Verma maam in partial fulfillment of the requirements for the course Problem Solving using C++.

Signature of Students: _____

Signature of Faculty: _____

Date: _____

Acknowledgment

we would like to express our sincere gratitude to our faculty Sakshi Verma ma’am for their guidance and support throughout this project. Their valuable suggestions helped me improve my understanding of C++ programming.

we also thank our institution for providing the necessary resources to complete this project. Lastly, we appreciate our friends and classmates for their support and encouragement.

ABSTRACT

The Library Management System is a C++ based application developed to manage basic library operations efficiently. The system addresses the problems of manual record-keeping such as time consumption, errors, and difficulty in tracking books.

The project uses Object-Oriented Programming concepts like classes and objects to represent book records. It provides functionalities such as adding books, searching books, issuing books, and returning books.

The system follows a menu-driven approach, making it simple and user-friendly. Although basic, the system demonstrates how real-world problems can be solved using programming techniques.

Table of Contents

1. Introduction
2. Objectives
3. System Requirements
4. Methodology
5. OOP Concepts Used
6. System Design
7. Implementation
8. Output
9. Features
10. Limitations
11. Future Scope
12. Conclusion
13. References

INTRODUCTION

A Library Management System is a software application designed to manage and organize library data efficiently. It helps in maintaining records of books and simplifies the process of issuing and returning books.

Manual systems are slow, error-prone, and difficult to maintain. This project aims to develop a computerized system using C++ that automates these operations and improves efficiency.

Objectives

- To develop a Library Management System using C++
- To implement Object-Oriented Programming concepts
- To reduce manual workload
- To improve efficiency in managing library records
- To provide quick access to book information

System Requirements

Hardware:

- Processor: Intel i3 or above
- RAM: Minimum 4GB

Software:

- Programming Language: C++
- Compiler: GCC / Turbo C++ / VS Code
- Operating System: Windows / Linux

Methodology / Design Approach

The project follows a menu-driven approach where users select operations from a list.

Approach Used:

- Bottom-up approach
- Modular programming

Structure:

- Class Book is created
- Functions perform operations
- Loop ensures continuous execution

OOP Concepts Used

Class & Object

A class Book is created and objects are used to store book details.

Encapsulation

Data and functions are combined inside the class.

Abstraction

Only necessary details are shown to the user.

Inheritance

(Not used in this basic version)

Polymorphism

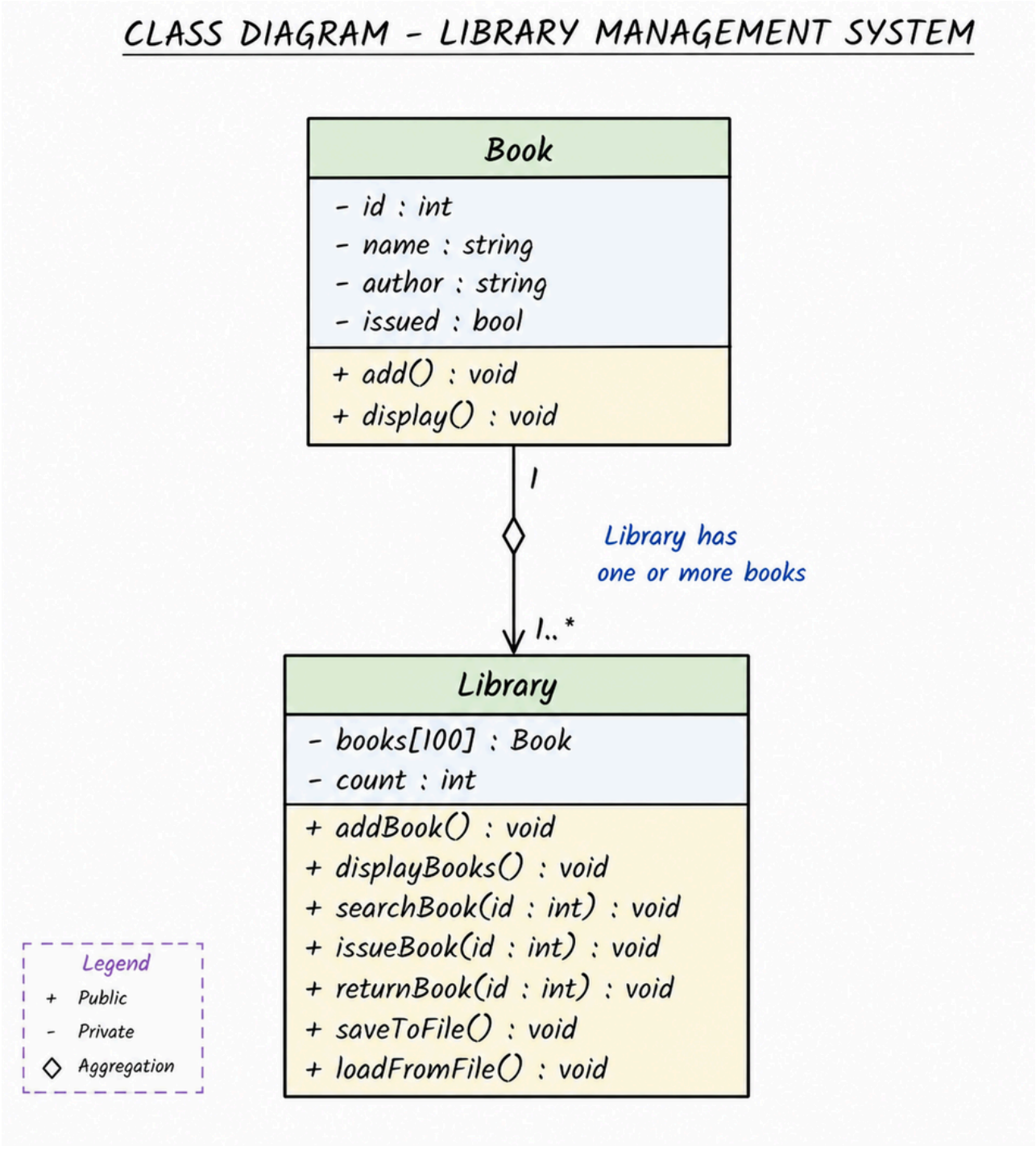
(Not used in this basic version)

Constructor / Destructor

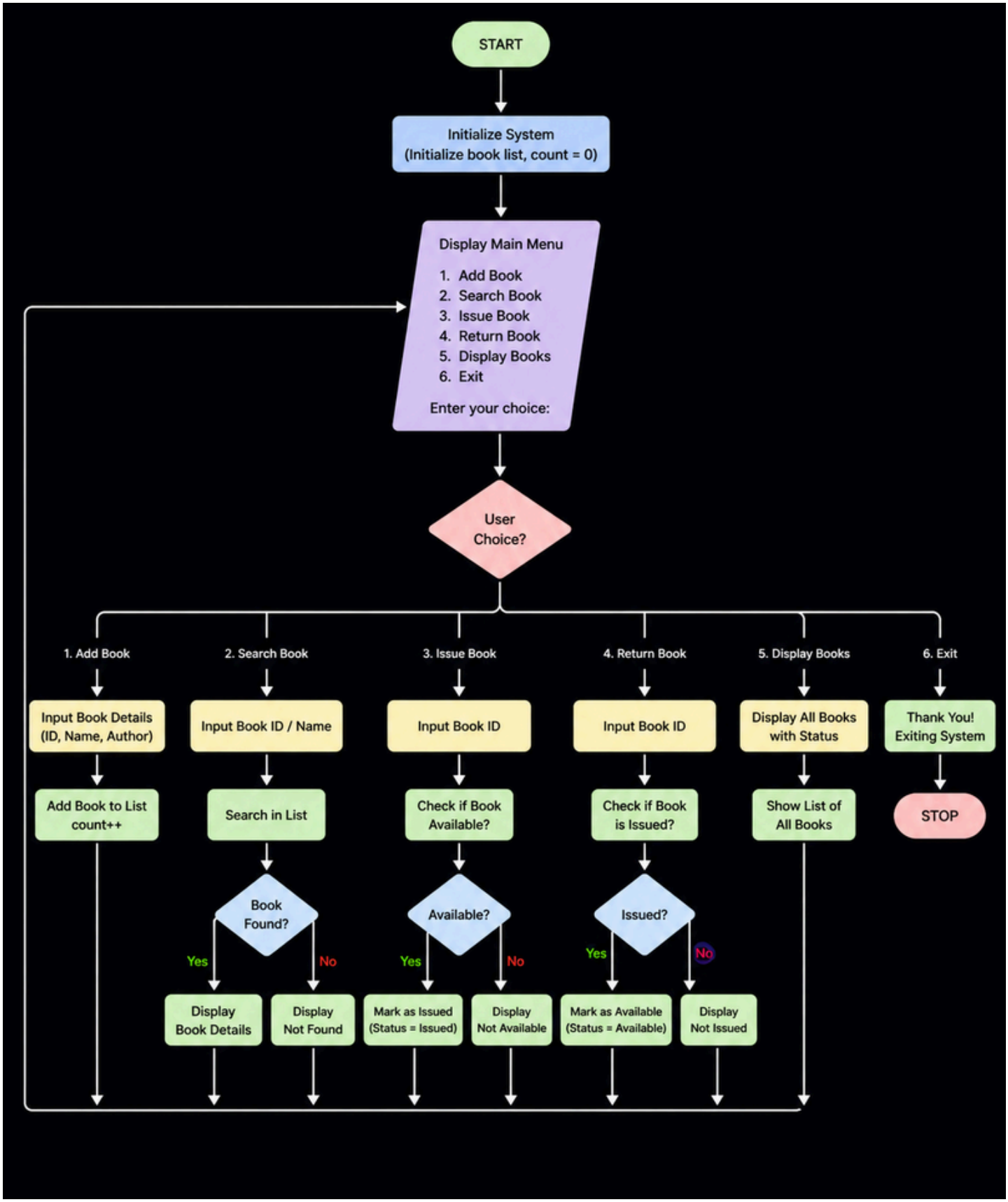
(Not implemented but can be added)

System Design

(a) Class Diagram



(b) flowchart


START
Initialize System
(Initialize book list, count = 0)
Display Main Menu
1. Add Book
2. Search Book
3. Issue Book
4. Return Book
5. Display Books
6. Exit
Enter your choice:
User Choice?
1. Add Book
2. Search Book
3. Issue Book
4. Return Book
5. Display Books
6. Exit
Input Book Details (ID, Name, Author)
Input Book ID / Name
Input Book ID
Input Book ID
Display All Books with Status
Thank You! Exiting System
Add Book to List count++
Search in List
Check if Book Available?
Check if Book is Issued?
Show List of All Books
STOP
Book Found?
Available?
Issued?
Yes
No
Yes
No
Yes
No
Display Book Details
Display Not Found
Mark as Issued (Status = Issued)
Display Not Available
Mark as Available (Status = Available)
Display Not Issued

Add Book:

1. Enter book details
2. Store in array
3. Mark as available

Search Book:

1. Enter ID
2. Compare with records
3. Display result

Issue Book:

1. Enter ID
2. Check availability
3. Mark as issued

Return Book:

1. Enter ID
2. Update status

Implementation (Code)

```cpp
#include <iostream>
#include <string>
using namespace std;

// 🔹 Class to represent a Book
class Book {
public:
    int id;             // Book ID
    string name;        // Book name
    string author;      // Author name
    bool issued = false; // Book status (false = available)

    // 🔹 Function to add book details
    void add() {
        cout << "Enter Book ID: ";
        cin >> id;
        cin.ignore(); // Clear input buffer

        cout << "Enter Book Name: ";
        getline(cin, name);

        cout << "Enter Author: ";
        getline(cin, author);

        issued = false; // New book is available
        cout << "Book added successfully!\n";
    }

    // 🔹 Function to display book details
    void show() {
        cout << "\nID: " << id
             << "\nName: " << name
             << "\nAuthor: " << author
             << "\nStatus: " << (issued ? "Issued" : "Available") << endl;
    }
};

int main() {
    Book b[100];  // Array to store up to 100 books
    int n = 0;    // Number of books currently stored
    int choice, id;
    bool found;   // Flag to check if book exists

    do {
        // 🔹 Display menu options
```

```
        // 🔹 Display menu options
        cout << "\n\n--- Library Menu ---\n";
        cout << "1. Add Book\n2. Display Books\n3. Search Book\n4. Issue Book\n5. Return Book\n6. Exit\n";
        cout << "Enter choice: ";
        cin >> choice;

        switch(choice) {

            // 🔹 Add a new book
            case 1:
                b[n].add(); // Call add function
                n++;        // Increase book count
                break;

            // 🔹 Display all books
            case 2:
                if(n == 0)
                    cout << "No books available.\n";
                else {
                    for(int i = 0; i < n; i++)
                        b[i].show(); // Display each book
                }
                break;

            // 🔹 Search for a book by ID
            case 3:
                cout << "Enter Book ID: ";
                cin >> id;
                found = false;

                for(int i = 0; i < n; i++) {
                    if(b[i].id == id) {
                        b[i].show(); // Show book details
                        found = true;
                        break;
                    }
                }

                if(!found)
                    cout << "Book not found.\n";
                break;

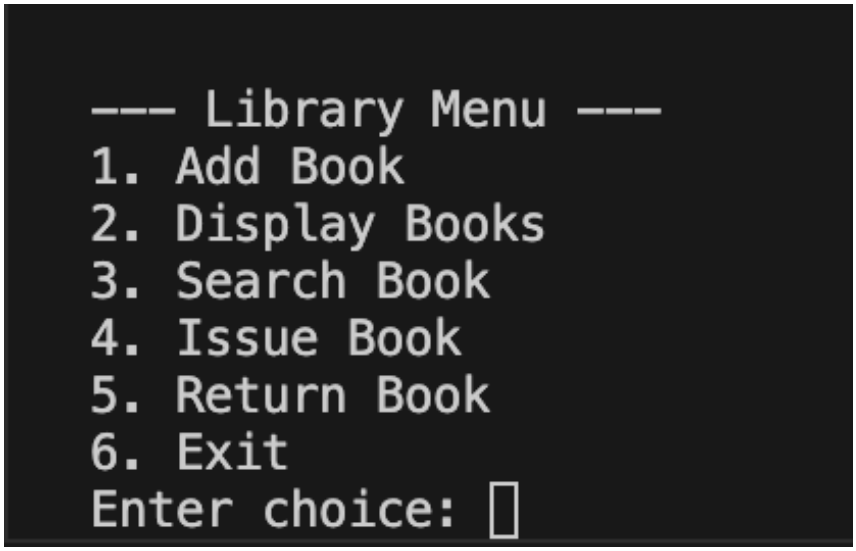
            // 🔹 Issue a book
            case 4:
                cout << "Enter Book ID to issue: ";
```

```
87          // 🔹 Issue a book
88          case 4:
89              cout << "Enter Book ID to issue: ";
90              cin >> id;
91              found = false;
92
93              for(int i = 0; i < n; i++) {
94                  if(b[i].id == id) {
95                      if(!b[i].issued) {
96                          b[i].issued = true; // Mark as issued
97                          cout << "Book issued successfully.\n";
98                      } else {
99                          cout << "Book already issued.\n";
100                     }
101                     found = true;
102                     break;
103                 }
104             }
105
106             if(!found)
107                 cout << "Book not found.\n";
108             break;
109
110         // 🔹 Return a book
111         case 5:
112             cout << "Enter Book ID to return: ";
113             cin >> id;
114             found = false;
115
116             for(int i = 0; i < n; i++) {
117                 if(b[i].id == id) {
118                     if(b[i].issued) {
119                         b[i].issued = false; // Mark as available
120                         cout << "Book returned successfully.\n";
121                     } else {
122                         cout << "Book was not issued.\n";
123                     }
124                     found = true;
125                     break;
126                 }
127             }
128
129             if(!found)
130                 cout << "Book not found.\n";
131             break;
```

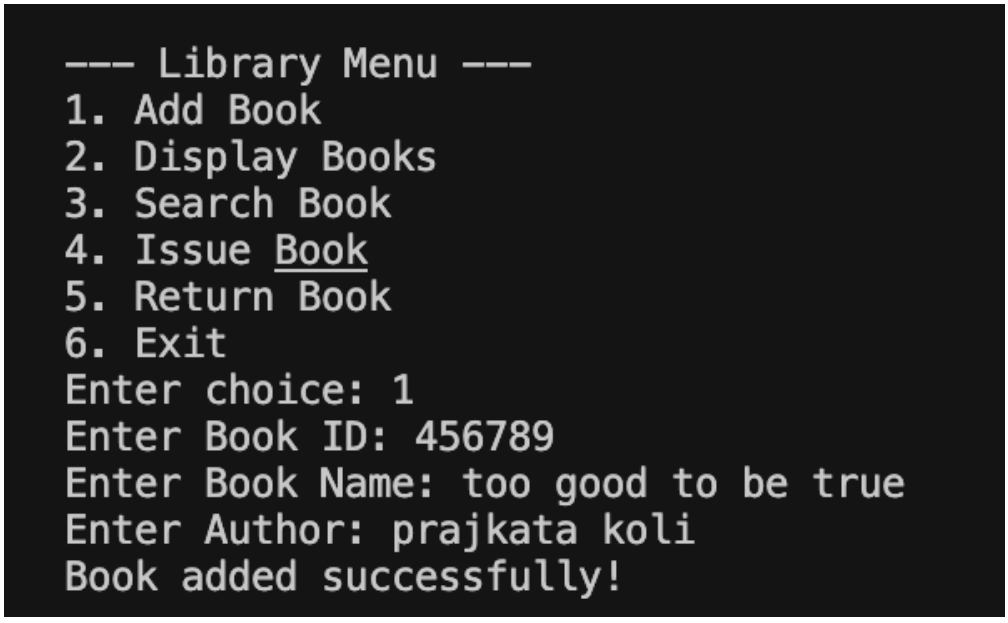
```
132
133             // 🔹 Exit program
134             case 6:
135                 cout << "Exiting...\n";
136                 break;
137
138             // 🔹 Invalid input handling
139             default:
140                 cout << "Invalid choice!\n";
141         }
142
143     } while(choice != 6); // Loop until user exits
144
145     return 0; // End of program
146 }
```

Output Screenshots / Results

Menu screen



Add book



- **Search book**

```
--- Library Menu ---
1. Add Book
2. Display Books
3. Search Book
4. Issue Book
5. Return Book
6. Exit
Enter choice: 3
Enter Book ID: 456789

ID: 456789
Name: too good to be true
Author: prajkata koli
Status: Available
```

- **Issue/Return**

```
--- Library Menu ---
1. Add Book
2. Display Books
3. Search Book
4. Issue Book
5. Return Book
6. Exit
Enter choice: 4
Enter Book ID to issue: 456789
Book issued successfully.
```

Features of the Project

- Add new books
- Display all books
- Search books by ID
- Issue books
- Return books
- Simple and user-friendly interface

Limitations

- No permanent data storage
- No graphical interface
- Limited to small-scale use
- No multi-user support

Future Scope

- Add file handling for data storage
- Develop graphical user interface
- Add login system
- Integrate database
- Add fine calculation

Conclusion

The Library Management System successfully demonstrates the use of C++ and Object-Oriented Programming concepts. It simplifies basic library operations and reduces manual work.

The project provides a strong foundation for developing more advanced systems in the future.

References

- C++ Programming Books
- Class Notes
- Online Tutorials (GeeksforGeeks, etc.)